

A restricted post is a post that can be viewed by the creator and the collaborators chosen by the creator. If there is no collaborator, only the creator can see the post.

We can break down the preceding rules to the following three conditions:

- Non-restricted posts are viewable by anyone
- Posts created by the currently logged-in user are viewable
- If a post is restricted, only the post' collaborators and creator can view it

From these three rules, we can see that they are related to the `index` and `show` actions in `PostsController`. We will pass an array of the `:index` and `:show` actions to the `can` method, because it takes an array of actions as the first argument, instead of just one action.

```
# the above rules for logged-in users
indexable_condition = <<-EOC
restricted = ? OR posts.user_id = ? OR
  (restricted = ? AND EXISTS
    (SELECT * FROM collaborations WHERE
      collaborations.post_id = posts.id AND
      collaborations.user_id = ?))
EOC

can [:index, :show], Post, [indexable_condition,
  false, user.id, true, user.id] do |post|
  !post.restricted? || post.user_id == user.id ||
    (post.restricted? &&
     post.collaborations.where(
       user_id: user.id).present?)
end
```

Although this way of using the CanCan ability is not the best way, it shows how you can leverage CanCan abilities to express complex SQL-based rules.

To display the list of posts that match the currently logged-in user's ability in the `index` action of `PostsController`, assign `Post.accessible_by(current_ability, :index)` to `@posts` instead of assigning `Post.all`.

With custom JS and CSS provided in the sample project, we can see a list of posts that are viewable by the currently logged-in user like this. Note that we can apply view checks on each of the buttons shown (**View**, **Edit**, and **Delete**).

www.it-ebooks.info